VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA", BELAGAVI – 570018, KARNATAKA



A Synopsis Report

On

"The Multimodal Touchless Human-Computer Interaction Engine"

In the partial fulfillment of the requirement for the award of degree

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Darshan M K	**4VZ23CS011**
Disha H	**4VZ23CS012**
Gagan	**4VZ23CS013**
Shreya M S	**4VZ23CS050**

Under the guidance of

Mrs. Sneha D. P

Assistant Professor,

Dept. of Computer Science and Engineering

VTU, Regional Office, Mysore



VISVESVARAYA TECHNOLOGIVCAL UNIVERSITY

Centre for Post Graduate studies, Sathagalli, Mysuru – 570029

2025-26

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA", BELAGAVI - 570018, KARNATAKA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Centre for Post Graduate Studies, Sathagalli, Mysuru – 570029.
2025 – 2026

CERTIFICATE

This is to certify that the Project work entitled, "The Multimodal Touchless Human-Computer Interaction Engine" is a bonafied work carried out by **DARSHAN M K, DISHA H, GAGAN , SHREYA M S** bearing USN **4VZ23CS011, 4VZ23CS012, 4VZ23CS013, 4VZ23CS050** at Department of **Computer Science and Engineering, Visvesvaraya Technological University, Centre for Post Graduate Studies, Mysuru** in partial fulfilment for the award of **Bachelor of Technology in Computer Science and Engineering, Visvesvaraya Technological University, Belagavi during the academic year 2025-2026**. It is certified that all the corrections/suggestions indicated during Internal Assessment have been incorporated in the report. The Synopsis Project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the **Bachelor of Technology degree**.

Signature of the Guide
Mrs. SNEHA D P
Assistant Professor,
Dept. of CS&E,
VTU, CPGS Mysuru – 570029

Signature of the Program Coordinator
Dr. G. F. ALI AHAMMED
Program Coordinator
Dept. of CS&E
VTU, CPGS Mysuru – 570029

Name of the Examiners: **Signature with Date:**

1. _____
2. _____

ABSTRACT

The Multimodal Touchless Human-Computer Interaction Engine is a real-time system that replaces traditional input devices by fusing four input modalities hand gestures as primary cursor control, eye blink detection for clicking, facial expressions for supplementary commands, and voice commands for system control. The system utilizes MediaPipe for 21 hand landmarks and 468 facial landmark detection, OpenAI Whisper for speech recognition, and pyttsx3 for auditory feedback. An OCR-based screen reader reads aloud basic screen content, while Generative AI via Google Gemini and Ollama enables intelligent summarization and translation for content-rich applications. An automatic context-switching mechanism dynamically adapts gesture mappings based on active applications.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	Introduction	03 – 05
	1.1 Aim	
	1.2 Problem Statement	
	1.3 Motivation	
	1.4 Scope of the Project	
	1.5 Objectives	
2.	Literature Survey	06 –09
3.	System Analysis	10 –13
	3.1 Existing System	
	3.2 Limitations of Existing System	
	3.3 Proposed System	
	3.4 Advantages of Proposed System	
4.	System Requirements	14 –16
	4.1 Hardware Specification	
	4.2 Software Specification	
	4.3 Functional Requirements	
	4.4 Non-Functional Requirements	
5.	Methodology	17
6.	Expected Outcome	18
7.	Conclusion	19
8.	References	20 – 21

CHAPTER 1

INTRODUCTION

Traditional human-computer interaction relies heavily on physical input devices such as the mouse and keyboard, which have remained fundamentally unchanged for over four decades. These devices present significant limitations in terms of accessibility for individuals with physical disabilities, hygiene in sterile environments such as hospitals and operating theatres, and usability in industrial scenarios where hands-free operation is critical. With advancements in computer vision, deep learning, and generative AI, a Multimodal Touchless Human-Computer Interaction Engine offers a dynamic, real-time, and intelligent solution to bridge the gap between humans and computers without any physical contact. By combining gesture-based control with voice commands, facial expression analysis, eye blink detection, and AI-powered screen interaction, the system empowers all users — including those with physical disabilities to interact with computers naturally and efficiently across diverse environments including healthcare, education, industry, and everyday computing.

1.1 Aim of the Project

To develop a Multimodal Touchless Human-Computer Interaction Engine that enables complete computer control through hand gesture recognition, eye blink detection, facial expression analysis, and voice commands, integrated with an OCR-based screen reader and Generative AI for intelligent screen interaction, and an automatic context-switching mechanism that dynamically adapts to active applications.

1.2 Motivation

Despite rapid advancements in artificial intelligence and computer vision, the fundamental mode of human-computer interaction — the physical mouse and keyboard — has remained largely unchanged for decades. This dependency on physical devices creates significant challenges for individuals with motor disabilities, users in sterile or hazardous environments, and scenarios where hands-free operation is essential. The hearing and visually impaired communities face additional barriers in accessing standard computing interfaces, further widening the digital divide.

The emergence of real-time computer vision frameworks such as Google MediaPipe, state-of-the-art speech recognition models like OpenAI Whisper, and powerful Generative AI platforms like Google Gemini presents a transformative opportunity to reimagine human-computer interaction. A touchless, multimodal system that interprets natural human inputs — gestures, blinks, expressions, and voice — can make computing truly accessible, intuitive, and inclusive. The integration of OCR-based screen reading and AI-powered content summarization and translation further enhances productivity and accessibility for users across all ability levels and linguistic backgrounds.

1.3 Problem Statement

Physical input devices such as the mouse and keyboard impose significant constraints on human-computer interaction, particularly for individuals with physical disabilities, users in hygiene-sensitive environments like hospitals, and operators in industrial settings requiring hands-free control. Existing touchless interaction systems are predominantly single-modal, relying only on hand gestures or voice commands independently, and lack the intelligence to adapt to different application contexts dynamically. Furthermore, current systems do not integrate screen reading capabilities for visually impaired users, nor do they leverage Generative AI for intelligent content interaction such as summarization and translation. The absence of a unified, multimodal, context-aware, and AI-integrated touchless interaction system leaves a significant gap in accessible and intelligent human-computer interaction.

1.4 Scope of the Project

The Multimodal Touchless Human-Computer Interaction Engine focuses on developing a desktop application that enables complete computer control without physical devices, targeting accessibility, productivity, and intelligent screen interaction across diverse environments. The system supports four input modalities — hand gestures as the primary cursor control mechanism, eye blink detection for click operations, facial expression analysis for supplementary commands, and voice commands for system control and AI feature triggering. It includes an OCR-based screen reader for basic screens such as the desktop, settings, and file explorer, and Generative AI integration via Google Gemini and Ollama for

summarization and translation of content-rich screens such as web pages, PDFs, spreadsheets, and code editors. An automatic context-switching mechanism detects active applications and dynamically loads appropriate gesture profiles. The system is deployed as a desktop application on Windows, macOS, and Linux using Python, MediaPipe, OpenCV, and PyAutoGUI. The project excludes full browser extension deployment, mobile application support, and advanced multi-user collaboration features, but provides a strong foundation for future expansion into augmented reality interfaces and wearable device integration.

1.5 Objectives

- To develop an intelligent touchless human-computer interaction system using hand gesture recognition, eye blink detection, facial expression analysis, and voice commands for computer control.
- To implement an OCR-based screen reading system integrated with Generative AI techniques for text summarization, translation, and auditory feedback.
- To design and implement an automatic context-switching mechanism that detects active applications and dynamically adapts gesture mappings accordingly.

CHAPTER 2

LITERATURE SURVEY

Survey on Hand Gesture Recognition from Visual Input Linardakis et al. (2025), **[1]** presented a comprehensive survey on **hand gesture recognition** from visual input, focusing on applications in **sign language recognition**, **virtual reality**, and robotics. Their work analyzed **RGB**, **depth**, and video-based approaches while discussing benchmark datasets and **feature extraction methods**. The study highlighted challenges such as **real-time recognition**, **occlusion handling**, and **computational efficiency**. However, the survey identified limitations in achieving **robust generalization** across diverse users and environments.

From Vision to Voice: A Multi-Modal Assistive Framework for the Physically Impaired Bhat et al. (2025), **[2]** developed a **multimodal assistive framework** integrating **OCR**, **text-to-speech conversion**, and **multilingual translation** for physically impaired users. The system utilized **Tesseract OCR**, **OpenCV**, and **Google Text-to-Speech** to convert image-based text into speech in real time. Their framework improved **accessibility**, **intuitive interaction**, and multilingual support. Nevertheless, the framework required **internet-dependent APIs** for certain translation and speech functionalities.

Deep Learning Approaches for Continuous Sign Language Recognition Khan et al. (2025), **[3]** proposed a comprehensive review of **continuous sign language recognition systems** based on **deep learning techniques**. Their framework categorized **spatial**, **temporal**, and **alignment-based approaches** for continuous sign language processing. The survey emphasized the importance of **real-time processing**, **signer diversity**, and **naturalistic datasets** in improving recognition performance. However, challenges related to **computational complexity** and variability among signers remained unresolved.

AI Virtual Mouse Malik et al. (2025), **[4]** introduced an **AI-powered virtual mouse system** that utilized **computer vision** and **machine learning** for **touchless cursor control**. The proposed system employed **OpenCV**, **MediaPipe**, and **PyAutoGUI** for **gesture tracking**, clicking, scrolling, and navigation. Their implementation enhanced **human-computer interaction** while improving hygiene through **contactless operation**. Despite its advantages, the system faced difficulties under varying lighting conditions and complex backgrounds.

A Lightweight CNN Approach for Hand Gesture Recognition via GAF Encoding of A-Mode Ultrasound Signals Shangguan et al. (2025), **[5]** proposed a **lightweight parameter-free attention convolutional neural network** for **hand gesture recognition** using ultrasound signals. Their method transformed one-dimensional ultrasound signals into two-dimensional images using **Gramian Angular Field encoding**. The model achieved **high classification accuracy** while maintaining a **compact architecture** suitable for **real-time applications**. However, the approach required **specialized ultrasound acquisition hardware**, limiting broader applicability.

A Methodological and Structural Review of Hand Gesture Recognition Across Diverse Data Modalities Shin et al. (2024), **[6]** conducted a methodological review of **hand gesture recognition systems** across **RGB**, **depth**, **skeleton**, **EEG**, **EMG**, and **multimodal data modalities**. The survey examined advances in **feature extraction** and **deep learning approaches** while identifying challenges in **signer-independent systems** and **continuous gesture recognition**.

A Novel Hybrid Deep Learning Architecture for Dynamic Hand Gesture Recognition Hax et al. (2024), **[7]** proposed a **hybrid deep learning architecture** combining **convolutional neural networks** with **recurrent neural networks** for **dynamic hand gesture recognition**. Their model utilized **Inception-v3** and **LSTM layers** to classify gestures in realistic environments with improved recognition accuracy. However, unusual gestures and real-time transmission issues affected responsiveness.

A Systematic Review of Hand Gesture Recognition: An Update From 2018 to 2024 Hashi et al. (2024), **[8]** presented a systematic review of **vision-based**, **sensor-based**, and **hybrid hand gesture recognition systems**. The study evaluated **gesture representation**, **feature extraction**, and **classification accuracy** while identifying challenges in **continuous gesture recognition** and dataset limitations.

Design of Human–Computer Interaction Gesture Tracking Model Based on Improved PSO and KCF Algorithms He et al. (2024), **[9]** developed a **gesture tracking model** for **human-computer interaction** using **particle swarm optimization** and **kernel correlation filtering algorithms**. Their approach improved **gesture detection accuracy**, **tracking**

performance, and **real-time interaction** under monocular RGB cameras. However, complex gesture sequences remained computationally demanding.

Enhancing Gesture-Controlled Virtual Mouse and Virtual Keyboard Using AI Techniques Kotti et al. (2024), **[10]** designed a **gesture-controlled virtual mouse** and **virtual keyboard system** using **OpenCV**, **MediaPipe**, and **artificial intelligence techniques**. Their framework enabled **touchless interaction** through hand gesture recognition and virtual assistant integration. The system reduced dependency on physical devices and improved usability in smart teaching environments.

Real-Time Dynamic Gesture Recognition Method Based on Gaze Guidance Zhang et al. (2024), **[11]** proposed a **real-time gesture recognition framework** based on **gaze guidance** and **lightweight feature fusion networks**. Their method used **gaze tracking** and **skeletal feature extraction** to improve dynamic gesture segmentation accuracy. The proposed model achieved **high recognition accuracy** with minimal inference time in mixed reality environments.

Real-Time Security Risk Assessment From CCTV Using Hand Gesture Recognition Koca (2024), **[12]** introduced a **security risk assessment system** using CCTV footage and **hand gesture recognition** based on **deep learning** and **MediaPipe architecture**. The framework enabled **automated behavior analysis** and **risk detection** in surveillance environments. However, high-quality video streams were required for accurate performance.

Toward Intuitive 3D Interactions in Virtual Reality: A Deep Learning-Based Dual-Hand Gesture Recognition Approach Qi et al. (2024), **[13]** proposed a **deep learning-based dual-hand gesture recognition framework** for intuitive 3D interactions in virtual reality. Their model combined **continuous wavelet transformation**, **convolutional neural networks**, and **temporal convolution networks** for gesture classification. The framework improved **natural interaction** in VR systems while outperforming benchmark models.

Accuracy Enhancement of Hand Gesture Recognition Using CNN Park et al. (2023), **[14]** proposed a **CNN-based hand gesture recognition framework** using **Ultra-Wide Band radar signals** and **2D Fast Fourier Transform techniques**. Their method enhanced **recognition accuracy** and reduced learning time for gesture classification. However, the requirement of radar hardware increased deployment complexity.

Advancing Multimodal Fusion in Human-Computer Interaction Jagnade et al. (2023), **[15]** proposed a multimodal HCI framework integrating eye tracking, lips detection, speech recognition, and voice synthesis for intelligent cursor control and auditory feedback. The system used **OpenCV, Mediapipe, PyAutoGUI, SpeechRecognition, and pyttsx3** to enable hands-free cursor movement, clicking, scrolling, and voice feedback. Their work improved **accessibility, real-time interaction, and user experience, especially for physically impaired users**. However, challenges such as lighting conditions, calibration, facial variability, and environmental noise affected system accuracy and performance.

Few-Shot User-Definable Radar-Based Hand Gesture Recognition at the Edge Mauro et al. (2022), **[16]** introduced a **radar-based hand gesture recognition framework** using **meta-learning** and **convolutional neural networks** for **edge computing applications**. Their model enabled **few-shot gesture learning** with reduced computational cost and improved adaptability to new users. Nevertheless, specialized radar sensors limited large-scale deployment.

The literature survey highlights the significant growth of hand gesture recognition and assistive technologies using artificial intelligence, computer vision, and deep learning techniques. The reviewed studies focused on applications such as sign language recognition, virtual mouse systems, virtual reality interaction, surveillance, and accessibility support for physically impaired users. Various approaches using CNNs, RNNs, LSTMs, radar, ultrasound, and vision-based systems achieved improved accuracy and real-time performance. However, challenges such as lighting variations, computational complexity, hardware dependency, signer diversity, and real-time responsiveness still remain. Overall, the survey shows that AI-based gesture recognition systems are becoming more efficient, interactive, and suitable for real-world applications, while offering scope for further improvements in scalability and robustness.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing Systems

- **Single-Modal Gesture Systems:** Applications such as basic virtual mouse implementations using MediaPipe and OpenCV support only hand gesture input for cursor control, lacking eye blink, facial expression, and voice command integration, limiting their utility for users with hand mobility restrictions.
- **Voice-Only Assistants:** Systems like Dragon NaturallySpeaking and basic voice command tools provide voice-based computer control but lack visual gesture recognition, making them inadequate for environments where voice commands are impractical such as noisy settings or meetings.
- **Traditional Screen Readers:** Tools like NVDA and JAWS extract text from UI elements for visually impaired users but cannot describe images, icons, or visual content on screen, and do not integrate Generative AI for intelligent content summarization or translation.
- **Eye Gaze Systems:** Dedicated eye tracking hardware such as Tobii eye trackers provide high-precision gaze-based control but require expensive specialized hardware, making them inaccessible for general users and unsuitable for widespread deployment.
- **Single-Application Gesture Tools:** Gesture-based tools like Leap Motion controllers support gesture interaction within specific applications but cannot automatically switch gesture mappings based on the active application, requiring manual reconfiguration.
- **Basic AI Assistants:** Voice assistants like Cortana and Google Assistant provide AI-powered responses but are not integrated with real-time screen content analysis, gesture control, or adaptive application context switching.

3.2 Limitations of Existing Systems

- **Single Modality Restriction:** Most existing systems rely on a single input modality — either gestures, voice, or eye tracking — making them inaccessible when that specific modality is unavailable or unreliable.

- **No Context Awareness:** Existing gesture systems use fixed gesture-to-action mappings that do not adapt to the active application, requiring users to manually switch modes or remember different commands for different software.
- **Expensive Hardware Requirements:** High-precision eye tracking and gesture recognition systems require costly specialized hardware, limiting accessibility to general users and low-resource environments.
- **No AI-Powered Screen Interaction:** Existing touchless systems lack integration with Generative AI, preventing intelligent screen content understanding, summarization, and translation from visual inputs.
- **Limited Accessibility Support:** Current systems do not provide dedicated accessibility profiles for users with varying physical abilities such as eyes-only or voice-only control modes with auditory feedback.
- **No OCR Screen Reading:** Existing touchless systems do not incorporate OCR-based reading of on-screen content, leaving visually impaired users without auditory screen descriptions in basic navigation scenarios.
- **Poor Performance on Standard Hardware:** Many advanced multimodal systems require dedicated GPUs for real-time processing, making them impractical for deployment on standard consumer laptops and desktops.

3.3 Proposed System

The Multimodal Touchless Human-Computer Interaction Engine is a desktop-based, AI-driven system that addresses existing limitations through:

- **Four-Modal Input Fusion:** Integrates hand gesture recognition as primary cursor control, eye blink detection for click operations, facial expression analysis for supplementary commands, and voice commands for system control — providing redundant input pathways for reliable operation.
- **MediaPipe-Based Detection:** Utilizes Google MediaPipe for real-time detection of 21 hand landmarks and 468 facial mesh landmarks through a standard RGB webcam, enabling accurate gesture, blink, and expression recognition without specialized hardware.

- **Whisper-Powered Voice Control:** Employs OpenAI Whisper for speech-to-text conversion, supporting 99 languages with robustness to accents and background noise, enabling reliable voice command processing.
- **OCR Screen Reader:** Uses pytesseract for text extraction from screenshots of basic screens — desktop, settings, file explorer, and error dialogs — with pyttsx3 for auditory text-to-speech output, providing screen reading without Generative AI dependency.
- **Generative AI Integration:** Leverages Google Gemini API and locally deployed Ollama with LLaMA for intelligent summarization and translation of content-rich screens including web pages, PDFs, Excel spreadsheets, and code editors.
- **Automatic Context Switching:** Detects the active application window using pygetwindow and dynamically loads YAML-based gesture profiles tailored for specific applications such as presentations, media players, browsers, and code editors.
- **Auditory Feedback System:** Provides real-time voice feedback for every action, AI response, and screen reading output through pyttsx3, enhancing accessibility for visually impaired users.
- **Preprocessing Pipeline:** Applies coordinate normalization, Exponential Moving Average smoothing, and boundary clamping to ensure jitter-free, accurate cursor control across varying distances and lighting conditions.
- **Performance Dashboard:** Provides a real-time Streamlit-based dashboard displaying FPS, gesture accuracy, inference latency, and system resource utilization for performance monitoring and evaluation.

3.4 Advantages of Proposed System

- **Complete Touchless Control:** Enables full computer operation — cursor movement, clicking, scrolling, dragging, and keyboard simulation — without any physical device contact.
- **Multimodal Redundancy:** Four input modalities ensure uninterrupted operation even when one modality is unavailable, improving reliability and accessibility.
- **No Specialized Hardware:** Operates with a standard webcam and microphone available on any consumer laptop, making it accessible and cost-effective.

- **Intelligent Screen Interaction:** Generative AI integration enables on-demand summarization and translation of any on-screen content, significantly enhancing productivity.
- **Universal Accessibility:** Dedicated accessibility modes enable complete computer control for users with paralysis, limb disabilities, or visual impairments through alternative input combinations.
- **Adaptive Context Switching:** Automatic gesture profile switching across applications eliminates the need for manual reconfiguration, providing a seamless user experience.
- **Offline Capability:** Ollama with local LLaMA models ensures full AI functionality without internet dependency, suitable for privacy-sensitive and restricted environments.
- **Real-Time Performance:** Achieves 30+ FPS with sub-30ms latency on standard hardware without GPU dependency, ensuring smooth and responsive interaction.
- **Auditory Feedback:** Complete voice feedback for all actions and AI responses ensures users are always informed of system status, enhancing usability for all ability levels.

CHAPTER 4

SYSTEM REQUIREMENT SPECIFICATION

This chapter outlines the system requirements necessary to develop and deploy the Multimodal Touchless Human-Computer Interaction Engine. These specifications ensure optimal real-time performance, accessibility, and reliable AI-powered screen interaction.

4.1 Hardware Specification

- **Processor:** Minimum Intel Core i5 (8th Generation) or AMD Ryzen 5 equivalent; Recommended Intel Core i7 or AMD Ryzen 7 for optimal real-time AI inference performance.
- **RAM:** Minimum 8GB for basic gesture and OCR features; Recommended 16GB for Ollama local LLM and concurrent multimodal processing.
- **Storage:** Minimum 20GB free SSD storage for system, models, and libraries; Recommended 50GB for full Ollama model storage.
- **Webcam:** Minimum 720p resolution at 30 FPS for gesture and facial detection; Recommended 1080p at 60 FPS for improved landmark accuracy.
- **Microphone:** Built-in laptop microphone for basic voice command processing; Recommended external USB microphone for improved Whisper transcription accuracy.
- **GPU:** Not mandatory; Optional NVIDIA GTX 1650 or higher for accelerated model inference when available.
- **Display:** Minimum 1366×768 resolution for coordinate mapping; Recommended 1920×1080 Full HD display.
- **Network:** Stable internet connection for Google Gemini API access; Offline mode supported through Ollama local LLM deployment.

4.2 Software Specification

- **Programming Language:** Python 3.10 (recommended) for full compatibility with all AI and computer vision libraries.

- **Computer Vision:** MediaPipe 0.10.x for hand landmark and face mesh detection; OpenCV 4.8.x for webcam frame capture and image processing.
- **Mouse and Keyboard Control:** PyAutoGUI 0.9.x for OS-level cursor movement, click simulation, and keyboard events; pynput for low-level input monitoring.
- **Speech Recognition:** OpenAI Whisper (base model) for offline speech-to-text conversion; SoundDevice for real-time microphone audio capture.
- **Generative AI:** Google Generative AI SDK (google-generativeai) for Gemini API integration; Ollama with LLaMA 3.2 for fully offline local LLM deployment.
- **OCR:** pytesseract (Tesseract OCR engine) for text extraction from screenshots; easyocr as alternative for improved multilingual text recognition.
- **Text-to-Speech:** pyttsx3 for offline auditory feedback and screen reading.
- **Dashboard:** Streamlit for real-time performance monitoring; Plotly for interactive metrics visualization.
- **Utilities:** pygetwindow for active window detection; psutil for system resource monitoring; python-dotenv for API key management.
- **Operating System:** Windows 10/11, macOS 12+, or Ubuntu 20.04 LTS and above.

4.3 Functionality Requirements

- Detect and track 21 hand landmarks in real time through a standard webcam for cursor movement, click, scroll, and drag operations.
- Detect intentional eye blinks and winks using Eye Aspect Ratio calculation for left and right click simulation without cursor movement by gaze.
- Recognize facial expressions including smile, eyebrow raise, and mouth open for supplementary command execution.
- Transcribe voice commands using OpenAI Whisper and execute mapped system actions including scroll, open application, and AI feature triggers.
- Capture screenshots and extract text using OCR for auditory reading of basic screens including desktop, settings, and file explorer.
- Send screenshots of content-rich screens to Gemini or Ollama for intelligent summarization and translation with auditory output.

- Detect active application window every two seconds and dynamically load appropriate gesture-to-action profile from YAML configuration files.
- Apply Exponential Moving Average smoothing, coordinate normalization, and boundary clamping to all landmark coordinates for stable cursor control.
- Display real-time performance metrics including FPS, gesture accuracy, and latency on a Streamlit dashboard.

4.4 Non-Functional Requirements

- **Performance:** Gesture recognition and cursor control must operate at minimum 30 FPS with end-to-end latency below 30ms for smooth real-time interaction.
- **Accuracy:** Hand gesture recognition must achieve minimum 95% accuracy; eye blink detection minimum 95%; facial expression recognition minimum 90%.
- **Reliability:** System must maintain stable performance across varying lighting conditions, distances from webcam, and partial hand visibility.
- **Usability:** Interface must require no technical expertise, with intuitive gesture vocabulary and clear auditory feedback for all user levels.
- **Accessibility:** System must support eyes-only, face-only, and voice-only control modes for users with varying physical abilities.
- **Privacy:** All gesture and expression processing must occur locally on-device; Gemini API calls must use HTTPS encryption; API keys stored in environment variables.
- **Portability:** System must operate on Windows, macOS, and Linux without platform-specific modifications.
- **Scalability:** Plugin architecture must allow new gesture profiles and application integrations without modifying the core engine.
- **Offline Capability:** Core gesture control, OCR screen reading, voice commands (Whisper), and AI summarization (Ollama) must function without internet connectivity.

CHAPTER 5

METHODOLOGY

The Multimodal Touchless Human-Computer Interaction Engine employs a modular, layered architecture to deliver real-time touchless computer control and intelligent screen interaction. The system continuously captures webcam frames through OpenCV, feeding into parallel processing pipelines for hand tracking, facial analysis, and context monitoring, while OpenAI Whisper listens for voice commands through the microphone.

The hand tracking module uses MediaPipe's Palm Detection Model to locate the hand region, followed by the Hand Landmark Model to detect 21 three-dimensional keypoints. Euclidean distances between specific landmark pairs determine the active gesture — index fingertip controls cursor movement, pinch triggers left click, two-finger gesture triggers right click, fist activates drag mode, and wrist rotation controls scrolling. All coordinates pass through a preprocessing pipeline applying normalization, Exponential Moving Average filtering, boundary clamping, and screen coordinate mapping before dispatching to PyAutoGUI for OS-level mouse simulation.

The facial analysis module uses MediaPipe Face Mesh to detect 468 facial landmarks for simultaneous eye blink and expression analysis. Eye Aspect Ratio calculations detect intentional left and right winks for click simulation, while expression analysis monitors smile, eyebrow raise, and mouth movement for supplementary command execution.

For AI-powered screen interaction, gesture or voice triggers capture a screenshot routed based on active application context — basic screens pass through pytesseract OCR for text extraction and pyttsx3 auditory reading, while content-rich screens are sent to Google Gemini or Ollama for intelligent summarization or translation with auditory output.

The context switching module runs as a background thread using pygetwindow, detecting active application changes every two seconds and loading corresponding YAML gesture profiles for presentations, media players, browsers, and coding environments without manual intervention. A Streamlit-based dashboard concurrently displays real-time FPS, gesture accuracy, latency, and system resource metrics.

CHAPTER 6

EXPECTED OUTCOME

- A fully functional desktop application enabling complete computer control — cursor movement, clicking, scrolling, dragging, and keyboard operations — without any physical input device through hand gestures, eye blinks, facial expressions, and voice commands.
- An OCR-based screen reading system that audibly describes on-screen content of basic screens and a Generative AI-powered system that summarizes and translates content-rich screens including web pages, PDFs, spreadsheets, and code editors.
- An automatic context-switching mechanism that seamlessly adapts gesture-to-action mappings based on the active application, providing optimized control across diverse software environments.
- A real-time performance monitoring dashboard displaying FPS, gesture accuracy, latency, and system resource metrics demonstrating production-grade system performance.
- Comprehensive accessibility profiles enabling complete computer access for users with physical disabilities through alternative input combinations including eyes-only, face-only, and voice-only control modes.
- Increased awareness of AI-powered touchless interaction technology, demonstrating practical applications in healthcare, industrial automation, education, and accessibility domains.

CONCLUSION

The Multimodal Touchless Human-Computer Interaction Engine transforms the paradigm of human-computer interaction by addressing the fundamental limitations of physical input devices through intelligent fusion of hand gesture recognition, eye blink detection, facial expression analysis, and voice commands. Leveraging Google MediaPipe for real-time landmark detection, OpenAI Whisper for accurate speech recognition, pytesseract for OCR screen reading, and Google Gemini with Ollama for Generative AI intelligence, the system delivers a complete, adaptive, and accessible touchless computing experience on standard consumer hardware. The automatic context-switching mechanism ensures seamless gesture adaptation across diverse applications, while comprehensive accessibility profiles empower users with varying physical abilities to achieve full computer control. With its modular architecture, offline capability, and real-time performance, this system establishes a strong foundation for the future of human-computer interaction. Future enhancements could include augmented reality integration, wearable device support, and expanded multilingual voice command processing to further broaden the system's impact across global user communities.

REFERENCES

[1] M. Linardakis, I. Varlamis, and G. T. Papadopoulos, "Survey on Hand Gesture Recognition from Visual Input," IEEE Access, vol. 13, 2025.

[2] S. Bhat, P. Bhat, and S. V. Kolekar, "From Vision to Voice: A Multi-Modal Assistive Framework for the Physically Impaired," IEEE Access, vol. 13, 2025.

[3] A. Khan et al., "Deep Learning Approaches for Continuous Sign Language Recognition: A Comprehensive Review," IEEE Access, vol. 13, 2025.

[4] A. Malik et al., "AI Virtual Mouse," IEEE Conference Publication, 2025.

[5] Q. Shangguan et al., "A Lightweight CNN Approach for Hand Gesture Recognition via GAF Encoding of A-Mode Ultrasound Signals," IEEE TNSRE, vol. 33, 2025.

[6] J. Shin et al., "A Methodological and Structural Review of Hand Gesture Recognition Across Diverse Data Modalities," IEEE Access, vol. 12, 2024.

[7] D. R. T. Hax et al., "A Novel Hybrid Deep Learning Architecture for Dynamic Hand Gesture Recognition," IEEE Access, vol. 12, 2024.

[8] A. O. Hashi et al., "A Systematic Review of Hand Gesture Recognition: An Update From 2018 to 2024," IEEE Access, vol. 12, 2024.

[9] D. He, Y. Yang, and R. Wu, "Design of Human–Computer Interaction Gesture Tracking Model on Improved PSO and KCF Algorithms," IEEE Access, vol. 12, 2024.

[10] J. Kotti, B. Padmaja, and D. Deepa, "Enhancing Gesture-Controlled Virtual Mouse and Virtual Keyboard Using AI Techniques," Journal of Mobile Multimedia, vol. 20, no. 2, 2024.

[11] B. Zhang, W. Li, and Z. Su, "Real-Time Dynamic Gesture Recognition Method Based on Gaze Guidance," IEEE Access, vol. 12, 2024.

[12] M. Koca, "Real-Time Security Risk Assessment From CCTV Using Hand Gesture Recognition," IEEE Access, vol. 12, 2024.

[13] T. D. Qi et al., "Toward Intuitive 3D Interactions in Virtual Reality: A Deep Learning-Based Dual-Hand Gesture Recognition Approach," IEEE Access, vol. 12, 2024.

[14] G. Park, V. K. Chandrasegar, and J. Koh, “Accuracy Enhancement of Hand Gesture Recognition Using CNN,” IEEE Access, vol. 11, 2023.

[15] G. Jagnade et al., “Advancing Multimodal Fusion in Human-Computer Interaction,” *IEEE ICCCNT*, 2023.

[16] G. Mauro et al., “Few-Shot User-Definable Radar-Based Hand Gesture Recognition at the Edge,” IEEE Access, vol. 10, 2022.